

Методические указания к Лабораторной работе №2: Сетевое взаимодействие и параллелизм

1. Введение в сетевое программирование с сокетами

Сетевое программирование — это основа для создания распределенных приложений, которые обмениваются данными через сеть. В Python низкоуровневое сетевое взаимодействие реализуется с помощью модуля `socket`, который предоставляет доступ к интерфейсу сокетов BSD. Сокеты являются конечными точками связи, через которые программы могут отправлять и получать данные.

1.1. Что такое сокеты?

Сокет (socket) — это абстрактная точка, через которую программа может отправлять или получать данные по сети. Сокеты позволяют приложениям взаимодействовать друг с другом, независимо от того, запущены ли они на одном компьютере или на разных машинах в сети.

Каждый сокет характеризуется:

- **Доменом (Address Family):** Определяет тип адресов, которые могут быть использованы. Наиболее распространены:
 - `socket.AF_INET`: Для IPv4-адресов (например, `127.0.0.1`).
 - `socket.AF_INET6`: Для IPv6-адресов.
 - `socket.AF_UNIX`: Для локальной связи между процессами на одном компьютере.
- **Типом сокета (Socket Type):** Определяет способ передачи данных.
 - `socket.SOCK_STREAM`: Для протокола TCP (Transmission Control Protocol). Обеспечивает надежную, упорядоченную, ориентированную на соединение передачу данных. Это то, что используется веб-браузерами, FTP и т.д.
 - `socket.SOCK_DGRAM`: Для протокола UDP (User Datagram Protocol). Обеспечивает быструю, но ненадежную, безгарантийную передачу данных без установления соединения. Используется для потокового видео, онлайн-игр.
- **Протоколом (Protocol):** Обычно равен 0, так как тип сокета уже подразумевает протокол (TCP или UDP).

1.2. Протокол TCP (Transmission Control Protocol)

TCP — это протокол, ориентированный на соединение. Это означает, что перед началом обмена данными между клиентом и сервером устанавливается логическое соединение. TCP гарантирует:

- **Надежность:** Данные доставляются без ошибок и потерь.
- **Упорядоченность:** Пакеты доставляются в том порядке, в котором были отправлены.
- **Контроль потока:** Предотвращает перегрузку получателя слишком большим объемом данных.
- **Контроль перегрузки:** Управляет объемом данных, отправляемых в сеть, чтобы избежать перегрузки сети.

Для TCP-соединения используются следующие шаги:

1. **Сервер:** Создает сокет, привязывает его к адресу и порту, начинает слушать входящие соединения.
2. **Клиент:** Создает сокет, подключается к адресу и порту сервера.
3. **Обмен данными:** После установления соединения клиент и сервер могут отправлять и получать данные.
4. **Закрытие соединения:** Соединение закрывается обеими сторонами.

1.3. Протокол UDP (User Datagram Protocol)

UDP — это протокол без установления соединения. Данные отправляются в виде отдельных пакетов (датаграмм) без гарантии доставки, порядка или отсутствия дубликатов. UDP быстрее TCP, но требует от приложения самостоятельной обработки надежности, если она нужна.

Для UDP-соединения:

1. **Сервер/Клиент:** Создает сокет, привязывает его к адресу и порту (для сервера или для клиента, если он ожидает ответ).
2. **Обмен данными:** Отправка и получение датаграмм без предварительного установления соединения.

2. Создание TCP-сервера

TCP-сервер — это программа, которая ожидает входящих соединений от клиентов, обрабатывает их запросы и отправляет ответы.

2.1. Основные шаги создания TCP-сервера

1. **Создание сокета:** `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`.

2. **Привязка к адресу и порту (`bind`):** `server_socket.bind((host, port))`. Сервер привязывается к определенному IP-адресу и порту, чтобы клиенты могли его найти.
3. **Начало прослушивания (`listen`):** `server_socket.listen(backlog)`. Сервер переходит в режим ожидания входящих соединений. `backlog` — это максимальное количество необработанных соединений в очереди.
4. **Принятие соединения (`accept`):** `conn, addr = server_socket.accept()`. Этот метод блокирует выполнение программы до тех пор, пока не будет установлено новое клиентское соединение. Он возвращает новый сокет `conn` (для общения с клиентом) и адрес клиента `addr`.
5. **Обмен данными:** Использование `conn.recv(buffer_size)` для получения данных и `conn.sendall(data)` для отправки.
6. **Закрытие соединения:** `conn.close()`. После завершения обмена данными сокет клиента закрывается.
7. **Закрытие серверного сокета:** `server_socket.close()`. После завершения работы сервера.

Пример 1: Простой TCP-сервер (однопоточный)

Этот сервер будет принимать одно соединение, получать сообщение, отправлять его обратно и закрывать соединение.

```
import socket

HOST = '127.0.0.1' # Стандартный адрес Loopback (localhost)
PORT = 65432      # Порт для прослушивания (непривилегированные порты > 1023)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen() # Начинаем прослушивать входящие соединения
    print(f"Сервер запущен на {HOST}:{PORT}. Ожидание подключений...")
    conn, addr = s.accept() # Принимаем входящее соединение
    with conn:
        print(f"Подключено клиентом: {addr}")
        while True:
            data = conn.recv(1024) # Получаем данные от клиента (максимум 1024 байта)
            if not data: # Если данных нет, клиент отключился
                break
            print(f"Получено от клиента: {data.decode()}")
            conn.sendall(data) # Отправляем полученные данные обратно клиенту
        print(f"Соединение с {addr} закрыто.")
```

Объяснение:

- `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`: Создает TCP-сокет для IPv4.
- `s.bind((HOST, PORT))`: Привязывает сокет к указанному адресу и порту.
- `s.listen()`: Переводит сокет в режим прослушивания. По умолчанию `backlog` может быть равен 1.
- `s.accept()`: Блокирует выполнение до тех пор, пока клиент не подключится. Возвращает новый сокет `conn` для общения с клиентом и адрес `addr`.
- `conn.recv(1024)`: Получает до 1024 байт данных. Возвращает пустую байтовую строку, если клиент отключился.
- `data.decode()`: Декодирует полученные байты в строку (по умолчанию UTF-8).
- `conn.sendall(data)`: Отправляет все данные обратно клиенту.
- Конструкция `with` гарантирует автоматическое закрытие сокетов.

3. Создание TCP-клиента

TCP-клиент — это программа, которая инициирует соединение с сервером, отправляет ему запросы и получает ответы.

3.1. Основные шаги создания TCP-клиента

1. **Создание сокета:** `socket.socket(socket.AF_INET, socket.SOCK_STREAM)`.
2. **Подключение к серверу (`connect`):** `client_socket.connect((host, port))`. Клиент пытается установить соединение с указанным сервером.
3. **Обмен данными:** Использование `client_socket.sendall(data)` для отправки и `client_socket.recv(buffer_size)` для получения.
4. **Закрытие соединения:** `client_socket.close()`.

Пример 2: Простой TCP-клиент

Этот клиент подключится к серверу из Примера 1, отправит сообщение и получит ответ.

```
import socket

HOST = '127.0.0.1' # IP-адрес сервера
PORT = 65432      # Порт сервера

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
```

```
s.connect((HOST, PORT)) # Подключаемся к серверу
message = "Привет, сервер!"
s.sendall(message.encode()) # Отправляем сообщение, закодированное в байты
data = s.recv(1024) # Получаем ответ от сервера

print(f"Получено от сервера: {data.decode()}")
```

Объяснение:

- `s.connect((HOST, PORT))`: Устанавливает соединение с сервером.
- `message.encode()`: Кодировает строку в байты (по умолчанию UTF-8) перед отправкой по сети.

4. Создание UDP-сервера и клиента

UDP-соединения проще, так как не требуют установления и разрыва соединения. Обмен данными происходит путем отправки датаграмм.

Пример 3: Простой UDP-сервер

```
import socket

HOST = '127.0.0.1'
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.bind((HOST, PORT))
    print(f"UDP-сервер запущен на {HOST}:{PORT}. Ожидание датаграмм...")
    while True:
        data, addr = s.recvfrom(1024) # Получаем данные и адрес отправителя
        print(f"Получено от {addr}: {data.decode()}")
        s.sendto(data, addr) # Отправляем данные обратно отправителю
```

Пример 4: Простой UDP-клиент

```
import socket

HOST = '127.0.0.1'
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    message = "Привет, UDP-сервер!"
    s.sendto(message.encode(), (HOST, PORT)) # Отправляем датаграмму
    data, addr = s.recvfrom(1024) # Получаем ответ
```

```
print(f"Получено от сервера: {data.decode()}")
```

Объяснение UDP-примеров:

- `socket.SOCK_DGRAM`: Указывает, что это UDP-сокеты.
- `s.recvfrom(1024)`: Получает датаграмму и возвращает кортеж `(data, address)`.
- `s.sendto(data, address)`: Отправляет датаграмму по указанному адресу.

5. Многопоточность и многопроцессность для сетевых приложений

Однопоточный сервер, показанный в Примере 1, может обрабатывать только одно клиентское соединение за раз. Для одновременной обработки нескольких клиентов необходимо использовать механизмы параллелизма: многопоточность (`threading`) или многопроцессность (`multiprocessing`).

5.1. Многопоточность (`threading`)

Поток (thread) — это легковесный поток выполнения внутри одного процесса. Все потоки в одном процессе разделяют одно и то же адресное пространство памяти. Это делает их легкими для создания и переключения, но также накладывает ограничения из-за Global Interpreter Lock (GIL) в CPython, который позволяет выполнять только один поток Python-кода за раз.

Когда использовать `threading`:

- Для задач, связанных с вводом/выводом (I/O-bound), таких как сетевые операции, чтение/запись файлов. Пока один поток ждет ответа от сети, GIL освобождается, и другой поток может выполнять Python-код.
- Когда требуется совместное использование данных между задачами (потоки легко обмениваются данными).

Пример 5: TCP-сервер с многопоточностью

Этот сервер будет создавать новый поток для каждого входящего клиентского соединения, позволяя обрабатывать несколько клиентов одновременно.

```
import socket
import threading

HOST = '127.0.0.1'
PORT = 65432
```

```
def handle_client(conn, addr):
    print(f"[АКТИВНО] Подключено клиентом: {addr}")
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print(f"[ОТ {addr}] {data.decode()}")
            conn.sendall(data)
    print(f"[ОТКЛЮЧЕНО] Соединение с {addr} закрыто.")

def start_server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((HOST, PORT))
        s.listen()
    print(f"[ЗАПУЩЕНО] Сервер запущен на {HOST}:{PORT}. Ожидание подключений")
    while True:
        conn, addr = s.accept()
        # Создаем новый поток для обработки клиента
        thread = threading.Thread(target=handle_client, args=(conn, addr))
        thread.start()
        print(f"[АКТИВНЫЕ СОЕДИНЕНИЯ] {threading.active_count() - 1}") # -1

if __name__ == "__main__":
    start_server()
```

Объяснение:

- Функция `handle_client` инкапсулирует логику обработки одного клиента.
- `threading.Thread(target=handle_client, args=(conn, addr))`:
Создает новый поток, который будет выполнять функцию `handle_client` с переданными аргументами.
- `thread.start()`: Запускает выполнение потока.
- `threading.active_count()`: Возвращает количество активных потоков.
Мы вычитаем 1, чтобы не учитывать основной поток.

5.2. Многопроцессность (`multiprocessing`)

Процесс (process) — это независимая единица выполнения, которая имеет свое собственное адресное пространство памяти. Процессы не разделяют память напрямую, что исключает проблемы с GIL и позволяет использовать все ядра процессора для CPU-bound задач.

Когда использовать `multiprocessing`:

- Для задач, интенсивно использующих процессор (CPU-bound), таких как сложные вычисления, обработка изображений, видео.
- Когда требуется полная изоляция между задачами.

Пример 6: TCP-сервер с многопроцессностью

Этот сервер будет создавать новый процесс для каждого входящего клиентского соединения.

```
import socket
import multiprocessing
import os

HOST = '127.0.0.1'
PORT = 65432

def handle_client(conn, addr):
    print(f"[ПРОЦЕСС {os.getpid()}] Подключено клиентом: {addr}")
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print(f"[ПРОЦЕСС {os.getpid()} ОТ {addr}] {data.decode()}")
            conn.sendall(data)
    print(f"[ПРОЦЕСС {os.getpid()}] Соединение с {addr} закрыто.")

def start_server():
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.bind((HOST, PORT))
        s.listen()
        print(f"[ЗАПУЩЕНО] Сервер запущен на {HOST}:{PORT}. Ожидание подключений")
        while True:
            conn, addr = s.accept()
            # Создаем новый процесс для обработки клиента
            process = multiprocessing.Process(target=handle_client, args=(conn,
            process.start()
            # Важно закрыть соединение в родительском процессе после передачи е
            conn.close()
            print(f"[АКТИВНЫЕ ПРОЦЕССЫ] {len(multiprocessing.active_children())}")

if __name__ == "__main__":
    start_server()
```

Объяснение:

- `multiprocessing.Process(target=handle_client, args=(conn, addr))`: Создает новый процесс.
- `process.start()`: Запускает выполнение процесса.
- `conn.close()` в родительском процессе: Очень важно закрыть сокет `conn` в родительском процессе после того, как он был передан дочернему. Иначе сокет будет оставаться открытым в обоих процессах, что может привести к проблемам.
- `os.getpid()`: Возвращает ID текущего процесса.

5.3. Выбор между `threading` и `multiprocessing`

Критерий	<code>threading</code> (Потоки)	<code>multiprocessing</code> (Процессы)
Использование	I/O-bound задачи (сеть, дисковый ввод/вывод)	CPU-bound задачи (вычисления)
GIL (CPython)	Влияет (один поток Python-кода за раз)	Не влияет (каждый процесс имеет свой GIL)
Обмен данными	Легкий (общая память), но требует синхронизации	Сложный (IPC: очереди, пайпы, сокеты)
Создание	Быстрое, легкое	Медленное, тяжелое (копирование памяти)
Изоляция	Низкая (общая память)	Высокая (изолированные адресные пространства)
Отказоустойчивость	Отказ одного потока может повлиять на весь процесс	Отказ одного процесса не влияет на другие

5.4. Многопоточный парсинг данных с `concurrent.futures`

В Лабораторной работе №1 мы рассматривали парсинг данных с веб-сайтов и работу с API. Часто возникает необходимость обработать большое количество страниц или запросов к API. Выполнение этих операций последовательно может занять много времени. Здесь на помощь приходит многопоточность, позволяющая выполнять несколько запросов одновременно.

Модуль `concurrent.futures` предоставляет высокоуровневый интерфейс для асинхронного выполнения вызываемых объектов. Он абстрагирует детали управления потоками (или процессами), делая параллельное программирование значительно проще. Наиболее часто используются `ThreadPoolExecutor` для многопоточности и `ProcessPoolExecutor` для multiprocessing.

`ThreadPoolExecutor` идеально подходит для задач, связанных с вводом/выводом (I/O-bound), таких как сетевые запросы (веб-скрейпинг, запросы к API). В таких задачах большая часть времени тратится на ожидание ответа от внешних ресурсов, а не на активные вычисления. Во время ожидания Python GIL освобождается, позволяя другим потокам выполнять свой код.

Основные шаги использования `ThreadPoolExecutor`:

1. Импортировать `ThreadPoolExecutor` из `concurrent.futures`.
2. Определить функцию, которая будет выполняться в каждом потоке (например, функция для загрузки и парсинга одной страницы).

3. Создать экземпляр `ThreadPoolExecutor`, указав максимальное количество рабочих потоков (`max_workers`).
4. Использовать метод `submit()` для отправки задач в пул потоков. Он возвращает объект `Future`.
5. Использовать метод `as_completed()` для получения результатов по мере их готовности, или `map()` для применения функции к итерируемому объекту и получения результатов в том же порядке, в каком были отправлены задачи.

Пример 7: Многопоточный парсинг нескольких URL

Предположим, у нас есть список URL, которые нужно загрузить и извлечь из них заголовки. Мы будем использовать `requests` (из ЛР №1) для загрузки и `BeautifulSoup` (из ЛР №1) для парсинга.

```
import requests
from bs4 import BeautifulSoup
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def fetch_title(url):
    """Загружает страницу по URL и извлекает заголовок."""
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status() # Проверка на HTTP-ошибки
        soup = BeautifulSoup(response.text, 'html.parser')
        title = soup.title.string if soup.title else "Нет заголовка"
        print(f"Заголовок для {url}: {title}")
        return url, title
    except requests.exceptions.RequestException as e:
        print(f"Ошибка при загрузке {url}: {e}")
        return url, None

def main_threaded_parsing():
    urls = [
        "https://www.python.org",
        "https://www.google.com",
        "https://www.wikipedia.org",
        "https://www.github.com",
        "https://www.bing.com",
        "https://www.stackoverflow.com",
        "https://www.mozilla.org",
        "https://www.apple.com",
        "https://www.microsoft.com",
        "https://www.amazon.com"
```

```

]

start_time = time.time()
results = []

# Создаем пул из 5 рабочих потоков
with ThreadPoolExecutor(max_workers=5) as executor:
    # Отправляем задачи в пул и получаем объекты Future
    future_to_url = {executor.submit(fetch_title, url): url for url in urls}

    # Обрабатываем результаты по мере их готовности
    for future in as_completed(future_to_url):
        url = future_to_url[future]
        try:
            data = future.result()
            results.append(data)
        except Exception as exc:
            print(f"Задача для {url} сгенерировала исключение: {exc}")

end_time = time.time()
print(f"\nВсего обработано {len(results)} URL за {end_time - start_time:.2f}с")
# print("Результаты:", results)

# if __name__ == "__main__":
#     main_threaded_parsing()

```

Объяснение:

- `ThreadPoolExecutor(max_workers=5)`: Создает пул, который будет использовать до 5 потоков одновременно. Это позволяет загружать 5 страниц параллельно.
- `executor.submit(fetch_title, url)`: Отправляет функцию `fetch_title` с аргументом `url` в пул. Возвращает объект `Future`, который представляет результат выполнения задачи.
- `as_completed(future_to_url)`: Итерирует по объектам `Future` по мере их завершения, независимо от порядка их отправки. Это позволяет обрабатывать результаты сразу, как только они становятся доступными.
- `future.result()`: Возвращает результат выполнения функции `fetch_title`. Если функция сгенерировала исключение, `result()` повторно сгенерирует это исключение.

Использование `ThreadPoolExecutor` значительно сокращает время выполнения I/O-bound задач по сравнению с последовательным подходом, так как потоки не блокируют друг друга во время ожидания сетевых операций.

6. Отправка и получение электронных писем

Python предоставляет встроенные модули для работы с электронной почтой.

`smtplib` используется для отправки писем, а `poplib` или `imaplib` — для получения.

6.1. Отправка электронных писем с `smtplib`

Модуль `smtplib` реализует клиентский протокол SMTP (Simple Mail Transfer Protocol), который используется для отправки электронной почты. Для отправки письма вам понадобится доступ к SMTP-серверу (например, Gmail, Mail.ru, Yandex.ru или ваш собственный).

Пример 9: Отправка простого текстового письма

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

# Данные для подключения к SMTP-серверу
SMTP_SERVER = 'smtp.gmail.com' # Для Gmail
SMTP_PORT = 587                # Порт для TLS/STARTTLS
SENDER_EMAIL = 'your_email@gmail.com' # Ваш Email
SENDER_PASSWORD = 'your_app_password' # Пароль приложения (для Gmail)
RECEIVER_EMAIL = 'recipient@example.com' # Email получателя

def send_simple_email(subject, body):
    msg = MIMEText(body, 'plain', 'utf-8')
    msg['Subject'] = subject
    msg['From'] = SENDER_EMAIL
    msg['To'] = RECEIVER_EMAIL

    try:
        with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
            server.starttls() # Переводим соединение в защищенный режим TLS
            server.login(SENDER_EMAIL, SENDER_PASSWORD)
            server.send_message(msg)
        print("Письмо успешно отправлено!")
    except Exception as e:
        print(f"Ошибка при отправке письма: {e}")

# Пример использования
# if __name__ == "__main__":
#     send_simple_email("Тестовое письмо из Python", "Это простое текстовое сооф
```

Важные замечания для Gmail:

- Для использования Gmail SMTP вам, скорее всего, потребуется сгенерировать **пароль приложения** (App Password) вместо обычного пароля от аккаунта, если у вас включена двухфакторная аутентификация. Это можно сделать в настройках безопасности Google-аккаунта.
- Убедитесь, что ваш аккаунт Google разрешает "менее безопасные приложения" (Less secure app access) или используйте пароль приложения.

6.2. Отправка письма с HTML-содержимым и вложениями

Для более сложных писем, содержащих HTML-форматирование или вложения, используется класс `MIMEMultipart`.

Пример 8: Отправка HTML-письма с вложением

```
import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart
from email.mime.base import MIMEBase
from email import encoders
import os

# Данные для подключения к SMTP-серверу (те же, что и выше)
SMTP_SERVER = 'smtp.gmail.com'
SMTP_PORT = 587
SENDER_EMAIL = 'your_email@gmail.com'
SENDER_PASSWORD = 'your_app_password'
RECEIVER_EMAIL = 'recipient@example.com'

def send_html_email_with_attachment(subject, html_body, attachment_path=None):
    msg = MIMEMultipart()
    msg['Subject'] = subject
    msg['From'] = SENDER_EMAIL
    msg['To'] = RECEIVER_EMAIL

    # Добавляем HTML-часть
    msg.attach(MIMEText(html_body, 'html', 'utf-8'))

    # Добавляем вложение, если указано
    if attachment_path and os.path.exists(attachment_path):
        try:
            with open(attachment_path, 'rb') as attachment:
                part = MIMEBase('application', 'octet-stream')
                part.set_payload(attachment.read())
                encoders.encode_base64(part)
                part.add_header(
```

```

        'Content-Disposition',
        f'attachment; filename= {os.path.basename(attachment_path)}',
    )
    msg.attach(part)
except Exception as e:
    print(f"Не удалось прикрепить файл {attachment_path}: {e}")
    return # Прерываем отправку, если вложение не удалось

try:
    with smtplib.SMTP(SMTP_SERVER, SMTP_PORT) as server:
        server.starttls()
        server.login(SENDER_EMAIL, SENDER_PASSWORD)
        server.send_message(msg)
    print("HTML-письмо с вложением успешно отправлено!")
except Exception as e:
    print(f"Ошибка при отправке HTML-письма с вложением: {e}")

# Пример использования
# if __name__ == "__main__":
#     html_content = """
#     <html>
#         <body>
#             <p>Привет!</p>
#             <p>Это <b>HTML-письмо</b>, отправленное из Python.</p>
#             <p>С уважением,<br>Ваш Python-скрипт</p>
#         </body>
#     </html>
#     """
#     # Создайте тестовый файл, например, 'test_attachment.txt'
#     # with open('test_attachment.txt', 'w') as f:
#     #     f.write('Это тестовое вложение.')
#     # send_html_email_with_attachment("HTML-письмо с файлом", html_content, "t

```

6.3. Получение электронных писем (обзор)

Для получения электронных писем используются другие протоколы:

- **POP3 (Post Office Protocol 3):** Простой протокол для загрузки писем с сервера на локальную машину. Обычно письма удаляются с сервера после загрузки.
 - Модуль Python: `poplib`.
- **IMAP (Internet Message Access Protocol):** Более сложный протокол, который позволяет управлять письмами непосредственно на сервере

(читать, перемещать, удалять), не загружая их полностью. Это позволяет синхронизировать состояние почты между несколькими устройствами.

- Модуль Python: `imaplib`.

Работа с `poplib` и `imaplib` требует более глубокого изучения и выходит за рамки данной лабораторной работы, но студентам полезно знать об их существовании и назначении. Основные шаги включают подключение к серверу, аутентификацию, выбор почтового ящика и получение сообщений. Сообщения затем парсятся с помощью модуля `email` для извлечения заголовков, тела и вложений.

✓ 7. Практические задания

Задание 1: Многопоточный веб-скрейпер для сбора информации

Цель: Разработать многопоточный веб-скрейпер, который будет параллельно собирать информацию с нескольких веб-страниц, используя `requests` и `BeautifulSoup`.

- Выбор целевых URL:** Выберите список из 10-15 URL-адресов новостных сайтов, блогов или страниц с товарами (например, с Wikipedia, новостных агрегаторов, или сайтов с открытыми данными). Убедитесь, что выбранные сайты не блокируют автоматизированный парсинг и не требуют сложной аутентификации.
- Функция парсинга:** Создайте функцию `parse_page(url)`, которая:
 - Принимает URL в качестве аргумента.
 - Использует `requests` для загрузки содержимого страницы.
 - Использует `BeautifulSoup` для извлечения определенной информации (например, заголовок страницы, первые 2-3 абзаца текста, ссылки на другие статьи, цена товара, описание).
 - Возвращает словарь или объект с извлеченными данными.
 - Добавьте обработку ошибок (например, `requests.exceptions.RequestException` для проблем с сетью, `AttributeError` для отсутствующих элементов).
- Многопоточная обработка:**
 - Используйте `concurrent.futures.ThreadPoolExecutor` для параллельного выполнения функции `parse_page` для всех URL из вашего списка.
 - Установите разумное количество рабочих потоков (например, 5-10).

- Соберите результаты выполнения всех задач.
- 4. **Вывод результатов:** Выведите собранную информацию в консоль или сохраните ее в файл (например, CSV или JSON).
- 5. **Измерение производительности:** Измерьте время выполнения скрипта с использованием многопоточности и сравните его с временем выполнения того же скрипта в однопоточном режиме (просто вызовите функцию `parse_page` для каждого URL в цикле). Сделайте выводы о приросте производительности.

Подсказка: Для измерения времени используйте модуль `time` (`time.time()`).

Задание 2: Контрольные вопросы

1. Что такое сокет? Какие основные параметры определяют сокет?
2. В чем ключевые различия между протоколами TCP и UDP? Когда следует использовать каждый из них?
3. Опишите последовательность действий при создании TCP-сервера и TCP-клиента.
4. Как клиент отправляет данные на сервер и как сервер их получает? Какие методы для этого используются?
5. Что такое многопоточность (`threading`) и multiprocessing (`multiprocessing`) в Python?
6. Когда предпочтительнее использовать многопоточность, а когда — multiprocessing для сетевых приложений?
7. Какие потенциальные проблемы могут возникнуть при работе с сокетами (например, занятый порт, недоступный хост) и как их можно обработать?
8. Объясните концепцию "эхо-сервера" и его практическое применение.